

Mikroprocesorová technika

Prednáška č.3: Prerušená a časovače

prof. Ing. Ivan Virgala, PhD.

Technická univerzita v Košiciach
Strojnícka fakulta

Katedra priemyselnej automatizácie a mechatroniky
ARM-lab (Applied Robotics and Mechatronics Laboratory)



Prerušenia

Asynchrónne udalosti, vektory a obslužné rutiny

č.	vektor	zdroj udalosti
1	RESET	reset mikrokontroléra
2	INT0_vect	externé prerušenie INT0
3	INT1_vect	externé prerušenie INT1
4	PCINT0_vect	zmena na skupine PCINT0
5	PCINT1_vect	zmena na skupine PCINT1
6	PCINT2_vect	zmena na skupine PCINT2
7	WDT_vect	watchdog
8	TIMER2_COMPA_vect	zhoda Timer2 A
9	TIMER2_COMPB_vect	zhoda Timer2 B
10	TIMER2_OVF_vect	pretečenie Timer2
11	TIMER1_CAPT_vect	zачytenie Timer1
12	TIMER1_COMPA_vect	zhoda Timer1 A
13	TIMER1_COMPB_vect	zhoda Timer1 B

č.	vektor	zdroj udalosti
14	TIMER1_OVF_vect	pretečenie Timer1
15	TIMER0_COMPA_vect	zhoda Timer0 A
16	TIMER0_COMPB_vect	zhoda Timer0 B
17	TIMER0_OVF_vect	pretečenie Timer0
18	SPI_STC_vect	dokončený SPI prenos
19	USART_RX_vect	prijatý bajt USART
20	USART_UDRE_vect	prázdny dátový register USART
21	USART_TX_vect	dokončené odoslanie USART
22	ADC_vect	dokončený ADC prevod
23	EE_READY_vect	EEPROM pripravená
24	ANALOG_COMP_vect	analogový komparátor
25	TWI_vect	TWI/I ² C udalosť
26	SPM_READY_vect	pripravenosť zápisu do Flash



Definícia

Prerušenie je mechanizmus, pri ktorom hardvérová alebo vnútorná udalosť dočasne presmeruje vykonávanie programu na príslušnú obslužnú rutinu.

Charakter mechanizmu

Nejde o paralelný program. CPU vždy vykonáva jednu postupnosť inštrukcií, iba sa dočasne zmení adresa ďalšej vykonávanej inštrukcie.



Porovnanie dvoch spôsobov reakcie programu na udalosť na vstupnom pine alebo v periférii.

Cyklické čítanie stavu

- program opakovane číta stav registra vstupu,
- reakčný čas závisí od dĺžky hlavnej slučky,
- pri viacerých úlohách rastie oneskorenie reakcie,
- vhodné pre jednoduché a pomalé deje.

Obsluha prerušením

- udalosť nastaví príznak hardvérovo,
- CPU prejde na obslužnú rutinu prerušenia,
- vhodné pre krátke impulzy a presné časovanie,
- vyžaduje disciplinovanú štruktúru programu.



- ① Zdroj prerušenia nastaví príznak udalosti.
- ② Ak je povolené konkrétne prerušenie aj globálne prerušenia, CPU dokončí práve vykonávanú inštrukciu.
- ③ Na zásobník sa uloží návratová adresa.
- ④ Program skočí na adresu príslušného vektora prerušenia.
- ⑤ Vykoná sa obslužná rutina **ISR**.
- ⑥ Inštrukcia návratu z prerušenia obnoví beh hlavného programu.

Externé prerušenie je vyvolané definovanou zmenou alebo úrovňou logického signálu na vstupnom pine.

Externé udalosti

- INT0 na pine PD2,
- INT1 na pine PD3,
- prerušenia pri zmene stavu vybraných pinov.

Vnútorne udalosti periférií

- pretečenie alebo porovnanie časovača,
- dokončenie ADC prevodu,
- príjem alebo odoslanie cez USART,
- SPI, TWI, watchdog a ďalšie periférie.



Prerušenie	Portový pin	Arduino pin	Typické použitie
INT0	PD2	D2	tlačidlo, impulz zo snímača, jeden kanál enkodéra
INT1	PD3	D3	druhý kanál enkodéra, externý impulz, synchronizácia

Rozšírenie externých prerušení pomocou PCINT

ATmega328P má dva samostatné vstupy externého prerušenia, INT0 a INT1. Na ďalších GPIO pinoch možno použiť prerušenie pri zmene logickej úrovne pomocou systému PCINT, ktorý je organizovaný do skupín podľa portov.



Register	Úloha pri externom prerušení
EICRA	voľba spúšťacej podmienky pre INT0 a INT1: úroveň, ľubovoľná zmena, zostupná hrana alebo nábežná hrana
EIMSK	maskovací register; povoľuje alebo zakazuje konkrétne externé prerušenie INT0/INT1
EIFR	príznakový register; obsahuje príznaky INTF0 a INTF1 nastavené udalosťou
SREG	stavový register CPU; bit I určuje globálne povolenie všetkých prerušení



Register EICRA určuje, pri akej zmene alebo úrovni signálu na vstupe INT0 alebo INT1 sa vytvorí požiadavka na prerušenie.

ISC01	ISC00	Podmienka pre INT0
0	0	nízka logická úroveň
0	1	ľubovoľná zmena logickej úrovne
1	0	zostupná hrana
1	1	nábežná hrana

INT1

Pre vstup INT1 platí rovnaký princíp, pričom sa používajú bity ISC11 a ISC10.

Prerušenie na ostupnej hrane signálu INT0

```
1 EICRA |= (1 << ISC01);  
2 EICRA &= ~(1 << ISC00);
```

Význam nastavenia

Pre vstup INT0 platí:

$$ISC01 = 1, \quad ISC00 = 0$$

Prerušenie pri zostupnej hrane.

Alternatívny zápis

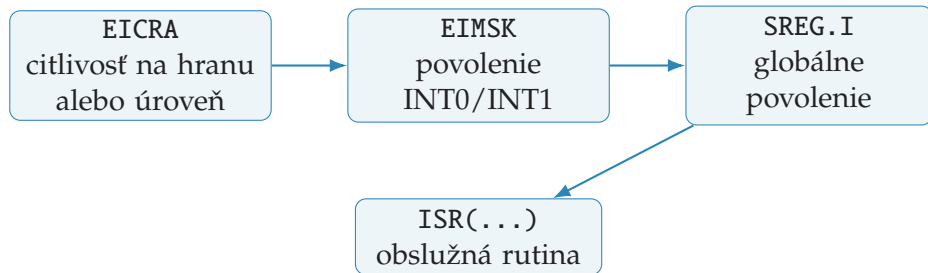
Ak je známy požadovaný obsah príslušných bitov: $EICRA = (1 \ll ISC01);$
Tento zápis však prepíše aj ostatné bity registra EICRA.



```
1 #include <avr/interrupt.h>
2
3 sei(); // nastaví I-bit v SREG
4 cli(); // vynuluje I-bit v SREG
```

Význam inštrukcií

- `sei()` globálne povolí prerušenia nastavením I-bitu v registri SREG,
- `cli()` globálne zakáže prerušenia vynulovaním I-bitu,
- konkrétne prerušenie musí byť súčasne povolené aj v príslušnom maskovacom registri.





```
1 // povolenie INT0
2 EIMSK |= (1 << INT0);
3
4 // vymazanie príznaku INTF0
5 EIFR |= (1 << INTF0);
6
7 // globálne povolenie prerušení
8 sei();
```

Logika povolenia

Prerušenie sa vykoná iba vtedy, keď je splnená kombinácia: nastala udalosť, príslušné prerušenie nie je maskované a globálny I-bit v SREG je nastavený.

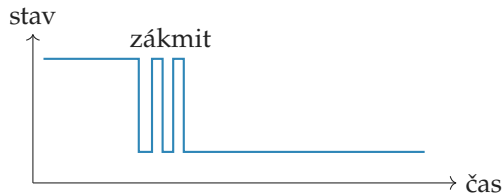


```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 ISR(INT0_vect)
5 {
6     // krátka obsluha udalosti
7 }
8
9 int main(void)
10 {
11     // konfigurácia GPIO a prerušenia
12     sei();
13
14     while (1) {
15         // hlavný program
16     }
17 }
```



```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 ISR(INT0_vect)
5 {
6     PORTB ^= (1 << PB5);    // prepnutie LED na PB5
7 }
8
9 int main(void)
10 {
11     DDRB |= (1 << PB5);    // PB5 ako výstup
12     DDRD &= ~(1 << PD2);  // PD2/INT0 ako vstup
13     PORTD |= (1 << PD2);  // interný pull-up
14
15     EICRA |= (1 << ISC01); // zostupná hrana INT0
16     EIMSK |= (1 << INT0);  // povolenie INT0
17     sei();
18
19     while (1) { }
20 }
```

Pri stlačení alebo uvoľnení mechanického tlačidla sa kontakt krátko rozkmitá, takže jeden fyzický úkon môže vytvoriť niekoľko rýchlych logických prechodov.



Dôsledok

Jedno fyzické stlačenie tlačidla môže vytvoriť viacero hrán a tým vyvolať viacero prerušení.

Softvérové potlačenie zákmitu

Po detekcii prvej hrany sa ďalšie zmeny vstupu počas krátkeho časového intervalu ignorujú. Po jeho uplynutí sa stav vstupu znovu overí. Typická doba potlačenia zákmitu je rádovo niekoľko až desiatky milisekúnd.

Hardvérové riešenie

Zákmit možno potlačiť napr. pomocou RC člena.

Ak je premenná menená v ISR a zároveň sa používa v hlavnom programe, jej hodnota sa môže zmeniť **asynchrónne**, bez priamej súvislosti s aktuálne vykonávanou časťou hlavného programu.

```
1 volatile uint8_t udalost = 0;
2
3 ISR(INT0_vect)
4 {
5     udalost = 1;
6 }
7
8 int main(void)
9 {
10     while (1)
11     {
12         if (udalost)
13         {
14             udalost = 0;
15             // spracovanie udalosti
16         }
17     }
18 }
```

Prečo volatile?

Bez `volatile` môže prekladač predpokladať, že sa hodnota premennej `udalost` v hlavnom programe nemení.

Pri optimalizácii by preto mohol hodnotu načítať iba raz a ďalej používať jej uloženú kópiu.

Kľúčové slovo `volatile` informuje prekladač, že hodnota sa môže zmeniť aj mimo aktuálne vykonávaného kódu. Pri každom použití sa preto musí čítať jej aktuálna hodnota z pamäte.



Praktické odporúčania

- ISR má byť čo najkratšia.
- Nepoužívame v nej dlhé oneskorenia ani blokujúce čakanie.
- Premenné spoločné pre ISR a hlavný program označujeme `volatile`.
- Zložitejšie spracovanie presúvame do hlavnej slučky.

Bezpečný prístup k viacbajtovým premenným

ATmega328P je 8-bitový mikrokontrolér, preto sa napríklad 16-bitová premenná číta po dvoch bajtoch. Ak ISR zmení jej hodnotu medzi čítaním prvého a druhého bajtu, môže vzniknúť nekonzistentný výsledok. Preto sa pri kritickom viacbajtovom prístupe na krátky čas zakáza prerušeniu a po dokončení operácie sa obnovia.



- Nastavené bity EICRA a EIMSK, ale chýba globálne povolenie `sei()`.
- Nesprávny pin: `INT0` je `PD2`, nie ľubovoľný digitálny pin.
- Použitie `_delay_ms()` vo vnútri ISR.
- Zabudnutý pull-up rezistor pri tlačidle.
- Neoznačená premenná `volatile`, hoci ju mení ISR.
- Ignorovanie zákmitu mechanického tlačidla.

Časovače a čítače

Hardvérové časovanie a periodické udalosti



Časovač

Počíta impulzy odvodené od hodinového signálu mikrokontroléra. Používa sa na meranie času, generovanie periodických udalostí a PWM.

Čítač

Počíta impulzy privedené z externého zdroja. Používa sa napríklad na počítanie hrán, impulzov alebo udalostí zo snímača.

Charakter periférie

Časovač beží hardvérovo. Počíta aj vtedy, keď CPU práve vykonáva inú časť programu.

Časovač	Šírka	Typické použitie
Timer/Counter0	8-bit	časovanie, počítanie externých impulzov, PWM
Timer/Counter1	16-bit	časovanie s väčším rozsahom, počítanie externých impulzov, Input Capture, PWM
Timer/Counter2	8-bit	časovanie, PWM, asynchrónna prevádzka s externým hodinovým zdrojom

Zameranie výkladu

Princíp bude vysvetlený na Timer0, pretože na ňom možno jednoducho ukázať preddelenie hodinového signálu, pretečenie a periodické prerušenie.



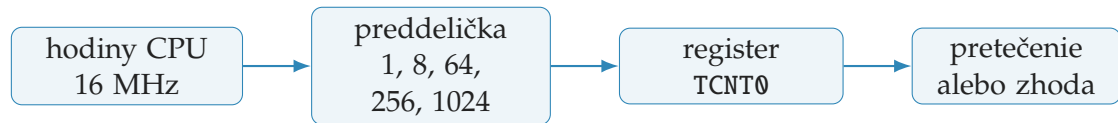
Definícia

Preddelička je hardvérový delič frekvencie. Z hodinového signálu CPU vytvorí pomalší takt, ktorým sa inkrementuje register časovača.

Úloha preddeličky

- zväčšuje čas medzi dvoma krokmi čítača,
- umožňuje dosiahnuť dlhšie časové intervaly,
- znižuje rozlíšenie časovača,
- volí sa podľa požadovaného rozsahu a presnosti.

$$T_{tick} = \frac{N}{f_{CPU}}, \quad f_{timer} = \frac{f_{CPU}}{N}$$



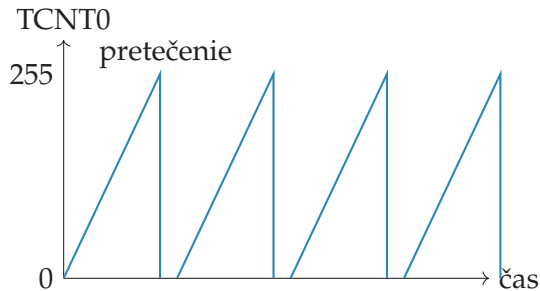
$$T_{tick} = \frac{N}{f_{CPU}}$$

N je hodnota preddeličky a f_{CPU} je taktovacia frekvencia mikrokontroléra.



Timer0 v normálnom režime

- register TCNT0 má 8 bitov,
- počíta od 0 po 255,
- po hodnote 255 nasleduje 0,
- tento okamih sa nazýva pretečenie,
- pri pretečení sa nastaví príznak TOV0.



$$T_{OVF} = 256 \cdot \frac{N}{f_{CPU}}$$

Preddelička N	T_{tick} pri 16 MHz	T_{OVF}
1	0,0625 μs	16 μs
8	0,5 μs	128 μs
64	4 μs	1,024 ms
256	16 μs	4,096 ms
1024	64 μs	16,384 ms

Príklad výpočtu

Pre interval približne 0,5 s a preddeličku 1024 je potrebných približne $0,5 / 0,016384 \approx 30,5$ pretečení.



Register	Význam
TCNT0	aktuálna hodnota 8-bitového čítača
TCCR0A	režim činnosti, výstupné porovnávacie módy
TCCR0B	výber hodinového zdroja a preddeličky, časť nastavenia režimu
TIMSK0	povolenie prerušení časovača
TIFR0	príznaky udalostí časovača
OCR0A, OCR0B	porovnávacie registre pre režim CTC alebo PWM

Bits CS02, CS01 a CS00 v registri TCCR0B určujú, z akého hodinového signálu bude Timer0 počítať.

```
1 // Timer0, preddeľička 1024
2 TCCR0B |= (1 << CS02) | (1 << CS00);
3
4 // povolenie prerušenia pri pretečení
5 TIMSK0 |= (1 << TOIE0);
```

CS02	CS01	CS00	Zdroj
0	0	1	bez delenia
0	1	0	/8
0	1	1	/64
1	0	0	/256
1	0	1	/1024



```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 volatile uint8_t overflow_count = 0;
5
6 ISR(TIMER0_OVF_vect)
7 {
8     overflow_count++;
9 }
```

Význam

Obslužná rutina sa vykoná vždy, keď TCNT0 prejde z hodnoty 255 späť na 0 a prerušenie pretečenia je povolené.



```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 volatile uint8_t ovf = 0;
5
6 ISR(TIMER0_OVF_vect)
7 {
8     ovf++;
9     if (ovf >= 31) {          // približne 0,5 s pri 16 MHz a /1024
10         PORTB ^= (1 << PB5);
11         ovf = 0;
12     }
13 }
14
15 int main(void)
16 {
17     DDRB |= (1 << PB5);
18     TCCR0B = (1 << CS02) | (1 << CS00); // /1024
19     TIMSK0 = (1 << TOIE0);
20     sei();
21     while (1) { }
22 }
```



Blokujúce čakanie

- jednoduché na prvé príklady,
- CPU počas čakania nevykonáva inú užitočnú úlohu,
- komplikuje reakciu na iné udalosti,
- nevhodné pre viac súbežných úloh.

Časovač

- čas plynie hardvérovo,
- hlavný program môže spracúvať inú časť algoritmu,
- reakcia môže byť periodická a presnejšia,
- vhodné pre stavové automaty a riadenie.



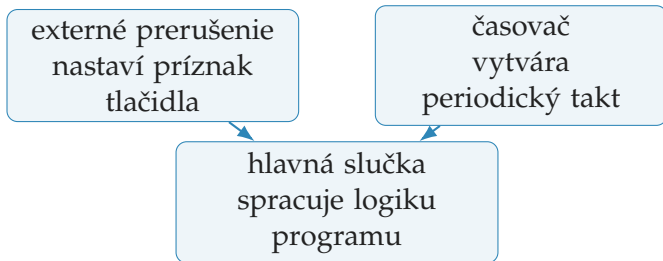
- V tomto predmete nepoužívame Arduino framework, preto registre časovačov nastavujeme priamo.
- Ak by sa použil Arduino framework, Timer0 býva obsadený funkciami ako `millis()` a `delay()`.
- Pri vlastnom programe v čistom C máme nad časovačmi plnú kontrolu.
- Nastavenie časovača môže ovplyvniť PWM výstupy pripojené na daný časovač.



- Zle vypočítaná hodnota preddeličky alebo OCR0A.
- Zabudnuté povolenie prerušenia v TIMSK0.
- Zabudnuté globálne povolenie sei().
- Príliš dlhá obsluha v ISR(TIMER0_..._vect).
- Použitie nesprávneho vektora prerušenia.

Architektúra embedded programu

Príznaky udalostí, hlavná slučka a periodická časová základňa



Princíp

ISR má len rýchlo zaznamenať, že nastala udalosť. Rozhodovanie a dlhšie operácie vykonáva hlavná slučka.



```
1 volatile uint8_t button_event = 0;
2
3 ISR(INT0_vect)
4 {
5     button_event = 1;
6 }
7
8 int main(void)
9 {
10     // konfigurácia ...
11     sei();
12
13     while (1) {
14         if (button_event) {
15             button_event = 0;
16             // spracovanie stlačenia tlačidla
17         }
18     }
19 }
```



Časovač Timer0 porovnáva aktuálnu hodnotu registra TCNT0 s hodnotou uloženou v registri OCR0A. Pri zhode $TCNT0 = OCR0A$ vznikne udalosť *Compare Match A*. Ak je toto prerušenie povolené, vykoná sa obslužná rutina `ISR(TIMER0_COMPA_vect)`.

```
1 volatile uint8_t tick_10ms = 0;
2 volatile uint8_t cnt_ms = 0;
3
4 ISR(TIMER0_COMPA_vect)    // vyvolaná každú 1 ms
5 {
6     cnt_ms++;
7
8     if (cnt_ms >= 10) {
9         cnt_ms = 0;
10        tick_10ms = 1;
11    }
12 }
```

Výsledok

Prerušenie každú 1 ms. Po 10 prerušeníach sa nastaví `tick_10ms`.



```
1 volatile uint8_t tick_10ms = 0;
2 volatile uint8_t button_event = 0;
3
4 ISR(INT0_vect) { button_event = 1; }
5 ISR(TIMER0_COMPA_vect) { tick_10ms = 1; }
6
7 int main(void)
8 {
9     // GPIO, INT0 a Timer0 konfigurácia
10    sei();
11    while (1) {
12        if (button_event) {
13            button_event = 0;
14            PORTB ^= (1 << PB5);
15        }
16        if (tick_10ms) {
17            tick_10ms = 0;
18            // periodická úloha
19        }
20    }
21 }
```



Zadanie

Navrhните program pre Arduino UNO R3 s ATmega328P:

- tlačidlo na PD2 / INT0 používa interný pull-up rezistor,
- LED je pripojená na PB5,
- stlačenie tlačidla prepne stav LED,
- Timer0 vytvára periodickú udalosť každých 10 ms,
- hlavný program počíta tieto udalosti,
- po uplynutí 10 s sa LED automaticky vypne,
- nové stlačenie tlačidla LED opäť zapne a začne nový 10-sekundový interval.

Ďakujem za
pozornosť